

# Customer Order Management System

## DBII Project Documentation

---

### Technologies Used

Python • Flask • SQLAlchemy • SQLite • HTML/CSS

*Database Systems II • 2025 / 2026*

## 1. Problem Statement

Small and medium-sized businesses often rely on spreadsheets or paper-based systems to track their customers, products, and orders. These approaches are error-prone, difficult to query, and do not scale as the business grows. The lack of a structured, relational data model makes it hard to answer even basic questions such as which customer has spent the most, which products are low on stock, or what the total revenue for a given period is.

This project addresses that problem by implementing a web-based Customer Order Management System built on a relational database. The system allows a business to:

- Register and manage customer records including contact information.
- Maintain a product catalogue with pricing and stock levels.
- Create orders that link customers to one or more products through order line items.
- Track order status (pending, completed, cancelled) and calculate revenue automatically.
- Analyse sales data through multi-table JOIN queries that combine information across all four tables.

The application demonstrates core relational database concepts including entity design, primary and foreign keys, one-to-many and many-to-many relationships, and SQL JOIN operations. The user interface is built with Flask and Jinja2, providing full CRUD (Create, Read, Update, Delete) functionality for all entities.

## 2. Entity-Relationship Model

### 2.1 Entities Overview

The system contains four entities. The table below summarises their type, primary key, and relationship to other entities.

| Entity      | Type    | Primary Key                                            | Notes                                                               |
|-------------|---------|--------------------------------------------------------|---------------------------------------------------------------------|
| Customers   | Strong  | id (PK)                                                | Exists independently. No dependency on other tables.                |
| Orders      | Strong* | id (PK), customer_id (FK)                              | Depends on Customers. Cannot have a valid order without a customer. |
| Products    | Strong  | id (PK)                                                | Exists independently. No dependency on other tables.                |
| Order Items | Weak    | id (PK surrogate), order_id + product_id (natural key) | Cannot exist without both an Order and a Product.                   |

## 2.2 Strong Entities

A strong entity has its own primary key and can exist independently of any other entity.

- **Customers:** Customers is a strong entity. A customer record exists on its own — it does not require an order or product to be created first. Its primary key is the auto-incrementing integer id.
- **Products:** Products is a strong entity. A product exists in the catalogue regardless of whether it has ever been ordered. Its primary key is the auto-incrementing integer id.

## 2.3 Weak Entity

A weak entity cannot exist without a parent entity. Its existence is dependent on one or more strong entities.

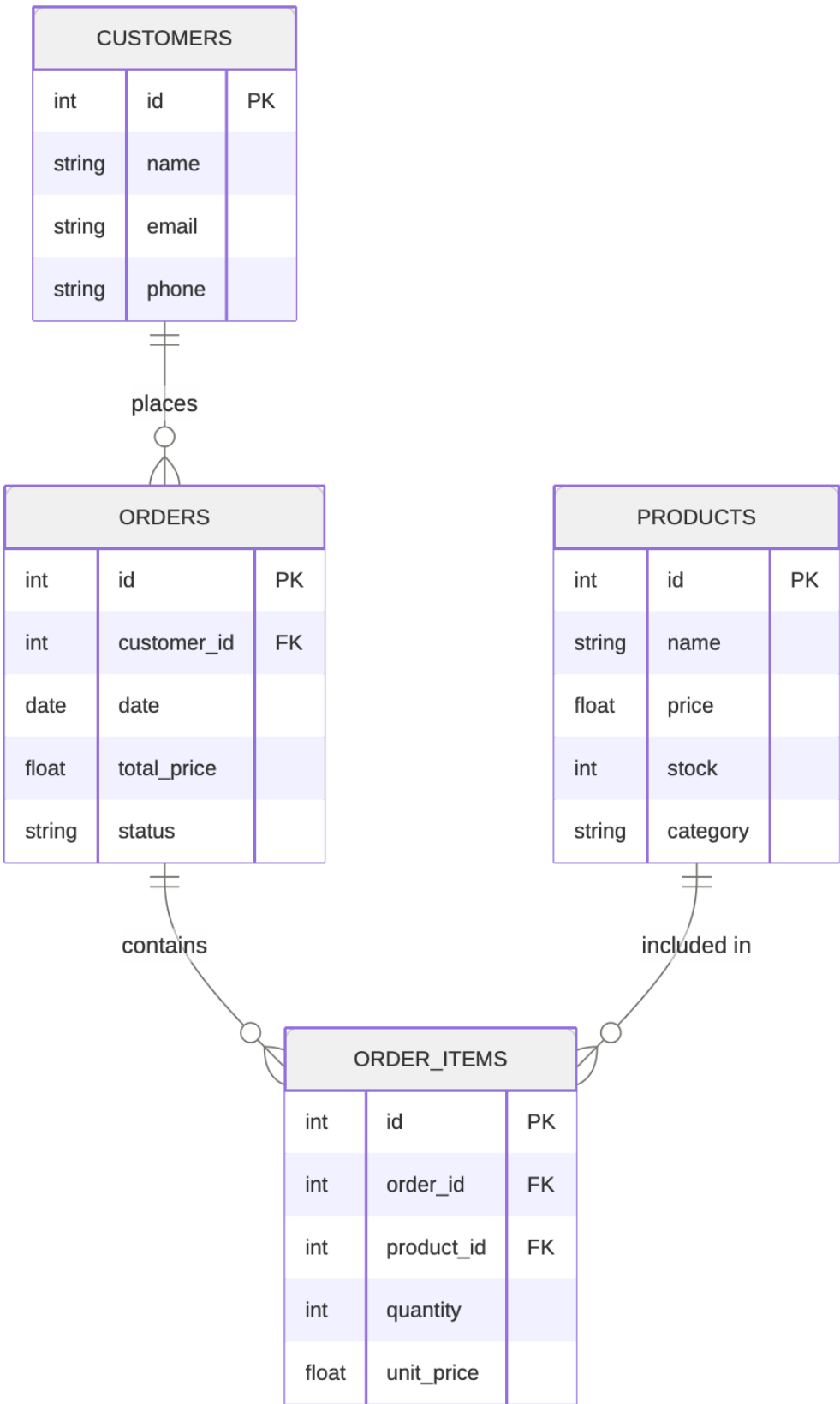
- **Order Items:** Order Items is the weak entity in this system. An order item record has no meaning unless both a parent Order and a parent Product exist. In strict ER theory, its natural (composite) key is the combination of order\_id and product\_id. A surrogate id column has been added for convenience, but the true identifier remains the composite foreign key pair.

## 2.4 Relationships

The diagram below shows the four entities and the relationships between them:

|           |        |             |                                                  |
|-----------|--------|-------------|--------------------------------------------------|
| CUSTOMERS | II--α{ | ORDERS      | (one customer places zero or many orders)        |
| ORDERS    | II--α{ | ORDER_ITEMS | (one order contains one or many line items)      |
| PRODUCTS  | II--α{ | ORDER_ITEMS | (one product appears in zero or many line items) |

The ORDER\_ITEMS table resolves the many-to-many relationship between ORDERS and PRODUCTS. A single order can contain many products, and a single product can appear in many orders. ORDER\_ITEMS sits between them, holding one row per product per order, along with the quantity and the unit price at the time of purchase.



## 3. Table Definitions

### 3.1 CUSTOMERS

Stores all customer contact information. This is a strong entity — it has no foreign keys and can exist independently.

| Column    | Type           | Constraint         | Description                                |
|-----------|----------------|--------------------|--------------------------------------------|
| <b>id</b> | <b>INTEGER</b> | <b>PRIMARY KEY</b> | <b>Auto-incrementing unique identifier</b> |
| name      | TEXT           | NOT NULL           | Full name of the customer                  |
| email     | TEXT           | NOT NULL, UNIQUE   | Email address — must be unique             |
| phone     | TEXT           | NULLABLE           | Optional contact phone number              |

### 3.2 ORDERS

Stores each order placed by a customer. Links to CUSTOMERS via a foreign key. One customer can have many orders (one-to-many).

| Column      | Type           | Constraint         | Description                                   |
|-------------|----------------|--------------------|-----------------------------------------------|
| <b>id</b>   | <b>INTEGER</b> | <b>PRIMARY KEY</b> | <b>Auto-incrementing unique identifier</b>    |
| customer_id | INTEGER        | FK -> customers.id | References the customer who placed this order |
| date        | DATE           | NOT NULL           | Date the order was placed                     |
| total_price | REAL           | NOT NULL           | Calculated total value of the order           |
| status      | TEXT           | DEFAULT 'pending'  | Order status: pending, completed, cancelled   |

The tables in this application are defined using SQLAlchemy's ORM layer, which automatically generates and executes the equivalent SQL at runtime. The raw SQL representation of all four tables is shown below for clarity:

```
CREATE TABLE IF NOT EXISTS customers (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    name TEXT NOT NULL,
```

```
    email TEXT NOT NULL UNIQUE,
```

phone TEXT

);

CREATE TABLE IF NOT EXISTS products (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT NOT NULL,

price REAL NOT NULL,

stock INTEGER DEFAULT 0,

category TEXT

);

CREATE TABLE IF NOT EXISTS orders (

id INTEGER PRIMARY KEY AUTOINCREMENT,

customer\_id INTEGER NOT NULL,

date TEXT NOT NULL,

total\_price REAL NOT NULL,

status TEXT DEFAULT 'pending',

FOREIGN KEY (customer\_id) REFERENCES customers(id)

);

CREATE TABLE IF NOT EXISTS order\_items (

id INTEGER PRIMARY KEY AUTOINCREMENT,

order\_id INTEGER NOT NULL,

product\_id INTEGER NOT NULL,

quantity INTEGER NOT NULL DEFAULT 1,

unit\_price REAL NOT NULL,

FOREIGN KEY (order\_id) REFERENCES orders(id),

FOREIGN KEY (product\_id) REFERENCES products(id)

);

### 3.3 PRODUCTS

Stores the product catalogue. This is a strong entity — it has no foreign keys and exists independently of orders.

| Column    | Type           | Constraint         | Description                                |
|-----------|----------------|--------------------|--------------------------------------------|
| <b>id</b> | <b>INTEGER</b> | <b>PRIMARY KEY</b> | <b>Auto-incrementing unique identifier</b> |
| name      | TEXT           | NOT NULL           | Name of the product                        |
| price     | REAL           | NOT NULL           | Current base price of the product          |
| stock     | INTEGER        | DEFAULT 0          | Current stock level in units               |
| category  | TEXT           | NULLABLE           | Optional product category for grouping     |

### 3.4 ORDER\_ITEMS (Weak Entity)

The junction table that resolves the many-to-many relationship between ORDERS and PRODUCTS. Each row represents one product line within one order. This is the weak entity — it cannot exist without both a parent Order and a parent Product.

| Column     | Type           | Constraint                     | Description                                         |
|------------|----------------|--------------------------------|-----------------------------------------------------|
| <b>id</b>  | <b>INTEGER</b> | <b>PRIMARY KEY (surrogate)</b> | <b>Surrogate key added for convenience</b>          |
| order_id   | INTEGER        | FK -> orders.id                | References the parent order                         |
| product_id | INTEGER        | FK -> products.id              | References the product being ordered                |
| quantity   | INTEGER        | NOT NULL                       | Number of units of this product in the order        |
| unit_price | REAL           | NOT NULL                       | Price per unit at time of purchase (price snapshot) |

## 4. JOIN Queries

---

### 4.1 JOIN 1 — Orders with Customer Details (2-table JOIN)

Purpose: Display every order alongside the name and email of the customer who placed it. This is a simple INNER JOIN between two tables.

```
SELECT
  orders.id      AS order_id,
  customers.name AS customer_name,
  customers.email AS customer_email,
  orders.date    AS order_date,
  orders.total_price AS total_price,
  orders.status  AS status
FROM orders
JOIN customers ON orders.customer_id = customers.id
ORDER BY orders.id DESC
```

How it works: The JOIN condition `orders.customer_id = customers.id` matches each order row to its corresponding customer row using the foreign key relationship. Only orders that have a matching customer are returned (INNER JOIN behaviour). The result set merges columns from both tables into a single flat result.

### 4.2 JOIN 2 — Order Item Detail (4-table JOIN)

Purpose: Show a full breakdown of every order line item — which customer ordered it, on which order, which product, at what quantity and price, and what the line total is. This query joins all four tables.

```
SELECT
  order_items.id      AS item_id,
  customers.name      AS customer_name,
  orders.id           AS order_id,
  orders.date         AS order_date,
  products.name       AS product_name,
  products.category   AS category,
  order_items.quantity AS quantity,
  order_items.unit_price AS unit_price,
  (order_items.quantity * order_items.unit_price) AS line_total
FROM order_items
JOIN orders  ON order_items.order_id = orders.id
JOIN customers ON orders.customer_id = customers.id
JOIN products ON order_items.product_id = products.id
ORDER BY orders.id DESC
```



How it works: The query chains three JOIN operations. Starting from ORDER\_ITEMS, it joins to ORDERS to get order metadata, then joins from ORDERS to CUSTOMERS to get the customer name, and finally joins from ORDER\_ITEMS to PRODUCTS to get the product name and category.

### 4.3 JOIN 3 — Product Sales Summary (LEFT JOIN + GROUP BY)

Purpose: Summarise sales performance per product — showing how many times each product was ordered, total units sold, and total revenue generated. Crucially, products that have never been ordered must still appear in the results with zero values.

```
SELECT
    products.name                AS product_name,
    products.category            AS category,
    products.price               AS unit_price,
    products.stock               AS stock,
    COUNT(order_items.id)        AS times_ordered,
    COALESCE(SUM(order_items.quantity), 0) AS total_units_sold,
    COALESCE(SUM(order_items.quantity * order_items.unit_price), 0) AS total_revenue
FROM products
LEFT JOIN order_items ON order_items.product_id = products.id
GROUP BY products.id
ORDER BY total_revenue DESC
```

How it works: A LEFT JOIN is used instead of an INNER JOIN so that products with no matching order items are still included in the result (they appear with NULL values that COALESCE converts to 0). The GROUP BY products.id collapses all order\_items rows for the same product into a single summary row. Aggregate functions COUNT and SUM operate over each group. The result is ordered by total\_revenue descending so the best-selling products appear first.

## 5. User Interface

The user interface is a web application built with Flask and rendered using Jinja2 HTML templates. All pages share a common layout defined in `base.html`, which provides a fixed sidebar navigation, a page title header, and a flash message system for user feedback. The application is styled with a custom dark theme.

### 5.1 Technology Stack

- **Backend:** Flask (Python) — web framework, routing, request handling
- **ORM:** SQLAlchemy — ORM for database interaction and schema definition
- **Database:** SQLite — lightweight file-based relational database (`app.db`)
- **Templates:** Jinja2 — server-side HTML templating with inheritance
- **Frontend:** HTML + CSS — custom dark theme with CSS variables

### 5.2 Application Pages

| Page               | Route                | Description                                                                   |
|--------------------|----------------------|-------------------------------------------------------------------------------|
| Dashboard          | /                    | Live stats: total customers, orders, and revenue. Shows 5 most recent orders. |
| Customers list     | /customers           | Table of all customers with order count, Edit and Delete actions.             |
| Add customer       | /customers/new       | Form to create a new customer record.                                         |
| Edit customer      | /customers/edit/<id> | Pre-filled form to update an existing customer.                               |
| Customer detail    | /customers/<id>      | Full profile of one customer with all their orders listed.                    |
| Orders list        | /orders              | All orders with customer name, date, total, and status badge.                 |
| Add / Edit order   | /orders/new          | Form with dynamic product rows and live total calculator.                     |
| Products list      | /products            | All products with colour-coded stock badges.                                  |
| Add / Edit product | /products/new        | Form to create or update a product.                                           |
| Reports            | /reports             | Three raw SQL JOIN query results displayed in separate tables.                |

### 5.3 CRUD Operations

The application implements all four CRUD operations for every entity:

- **Create (POST):** All records are retrieved from the database and displayed in paginated tables.

- **Read (GET):** New records are created via form submission using HTTP POST requests.
- **Update (POST):** Existing records can be modified through pre-filled edit forms.
- **Delete (POST):** Records are removed via POST requests with a browser-native confirmation dialog.

## 5.4 Template Inheritance

All pages extend `base.html` using Jinja2 template inheritance. The base template defines three blocks that child pages override:

- **{% block title %}:** Sets the browser tab title for each page.
- **{% block page\_title %}:** Sets the `<h1>` heading displayed in the top bar.
- **{% block content %}:** Contains the unique content of each page — tables, forms, or reports.

This approach eliminates code duplication. The sidebar, navigation, flash message system, and page structure are defined once and inherited by all ten templates.

## 5.5 Key Implementation Details

- SQLAlchemy relationships allow traversal between tables in Python without writing JOIN queries. For example, `order.customer.name` retrieves the customer name through the foreign key relationship automatically.
  - Flash messages provide one-time success, error, and info notifications after every create, update, and delete operation.
  - The order form uses JavaScript to add and remove product rows dynamically, and recalculates the order total in real time as the user selects products and quantities.
  - Deleting a customer automatically deletes all their orders and order items via SQLAlchemy cascade rules, maintaining referential integrity.
  - The `unit_price` column in `ORDER_ITEMS` stores a snapshot of the product price at the time of purchase, ensuring historical orders are not affected if a product price changes later.
-